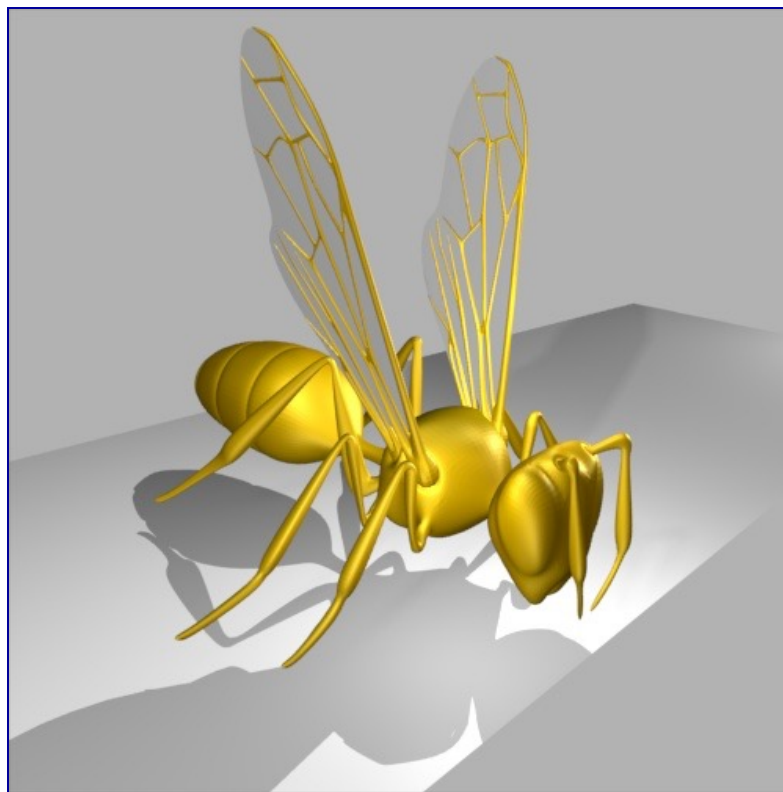


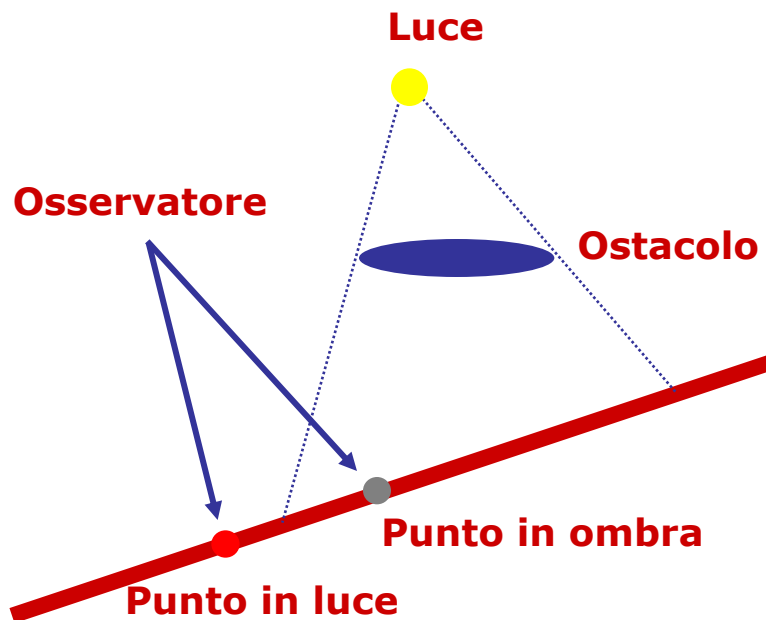


Shadowing



L'Ombra

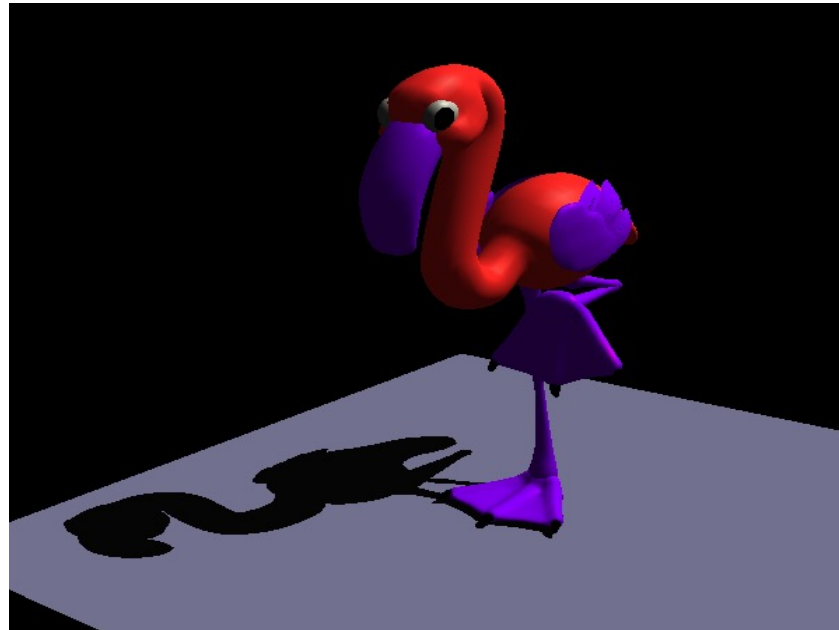
Un punto è in ombra se la luce non riesce a raggiungerlo perché bloccata da un ostacolo.



Per disegnare un pixel con "colore ombra" bisogna aver memorizzato l'informazione se il punto di cui il pixel è l'immagine sia esposto ad una luce o meno.

Funzioni dell'ombra

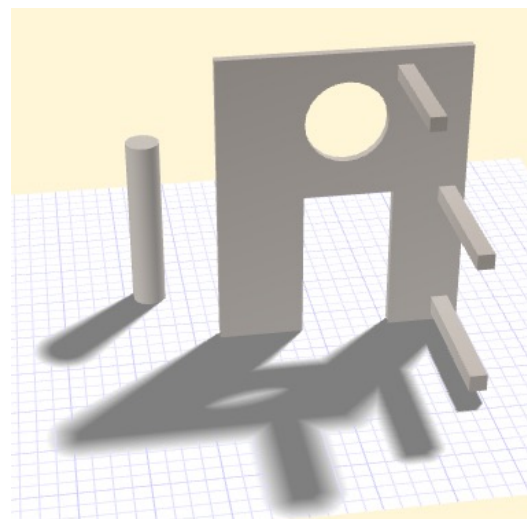
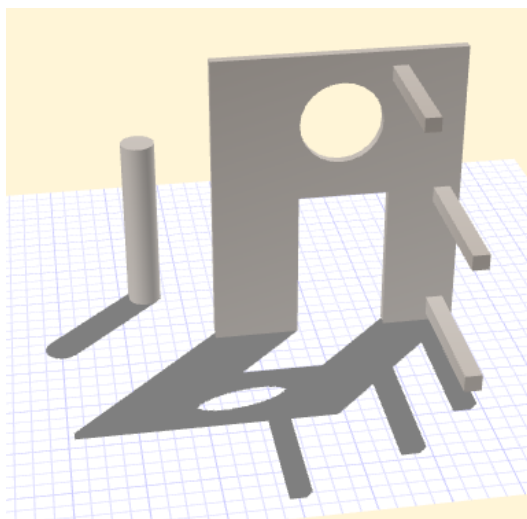
- Illustra se l'oggetto è appoggiato su una superficie o meno;
- Enfatizza la posizione della sorgente luminosa;



Ombra e Penombra

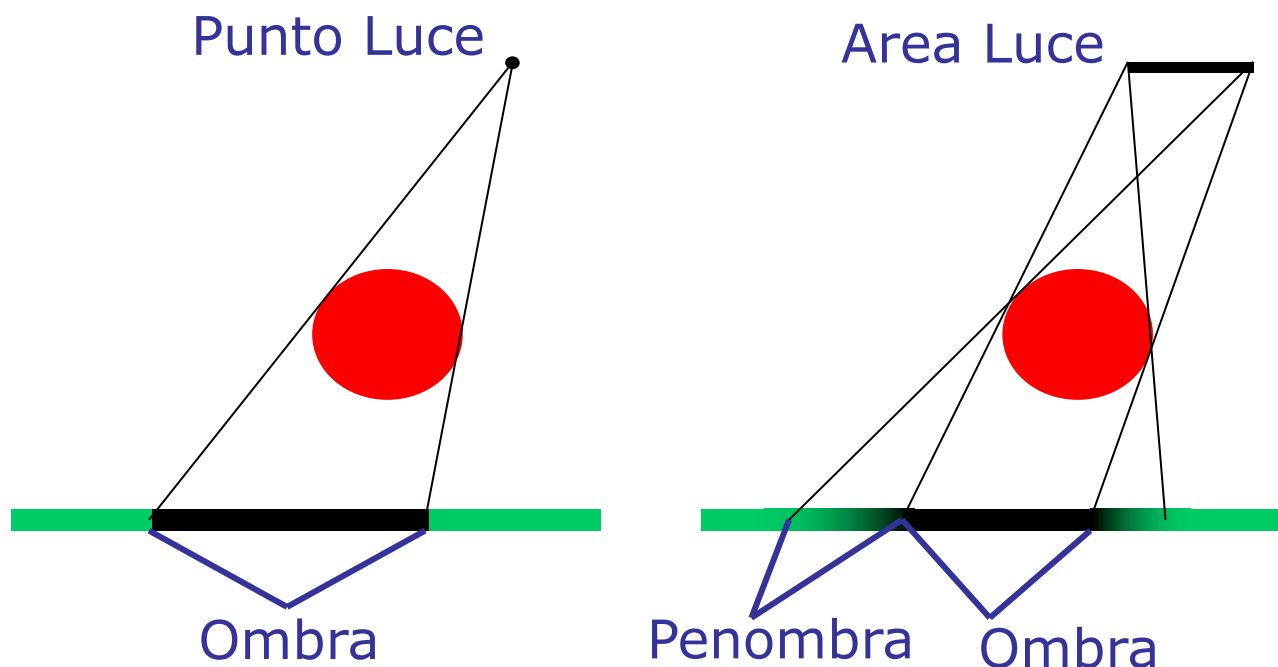
Le ombre variano moltissimo a seconda dell'ambiente luminoso. Possono avere contorni netti o contorni sfumati e contenere sia zone di **ombra** che di **penombra**

La dimensione relativa dell'ombra/penombra è una funzione della dimensione e forma della sorgente luminosa e della distanza dall'oggetto.



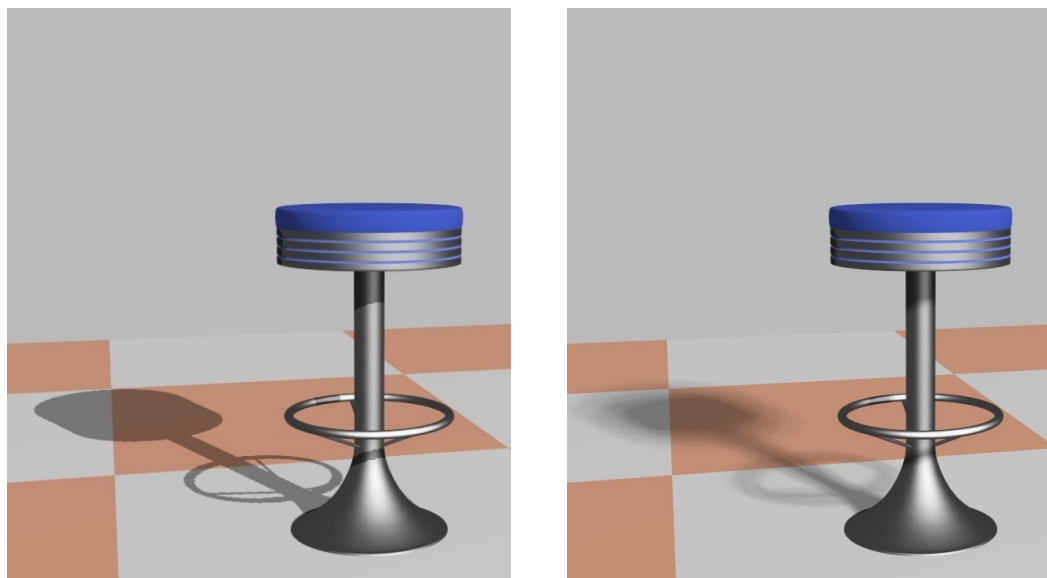
Ombra e Penombra

L'**ombra** è quella parte completamente non illuminata dalla sorgente, mentre la **penombra** è un'area che riceve una certa luce dalla sorgente; la penombra si sovrappone all'ombra e c'è sempre un graduale passaggio di intensità dall'una all'altra.



Ombra e Penombra

In Computer Graphics, solitamente si considera una **sorgente luminosa puntiforme** ad una certa distanza (o all'infinito) e ci si limita al caso più semplice di oggetti che producono un'ombra netta (**hard shadow**).



Anche in questa situazione di sorgente puntiforme, a causa dell'effetto diffrazione della luce dietro l'oggetto, l'ombra non dovrebbe avere dei contorni netti, ma gradualmente (**soft shadow**).



Osservazioni

- Se l'osservatore e la sorgente luminosa coincidono come posizione, non ci sono ombre;
- Per scene statiche, le ombre sono fisse e non cambiano al cambiare della posizione dell'osservatore;
- Se la posizione relativa dell'oggetto e della sorgente cambiano, le ombre devono essere ricalcolate. Questo porta ad un alto costo del rendering dove le ombre sono importanti per percepire la profondità e il movimento.
- Molti algoritmi di shadowing si occupano solo di modelli poligonal e questi producono ombre poligonal (con contorni poligonal).



Algoritmi di Shadowing

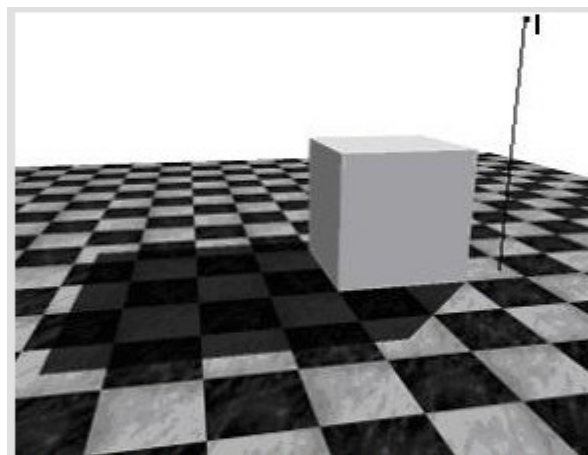
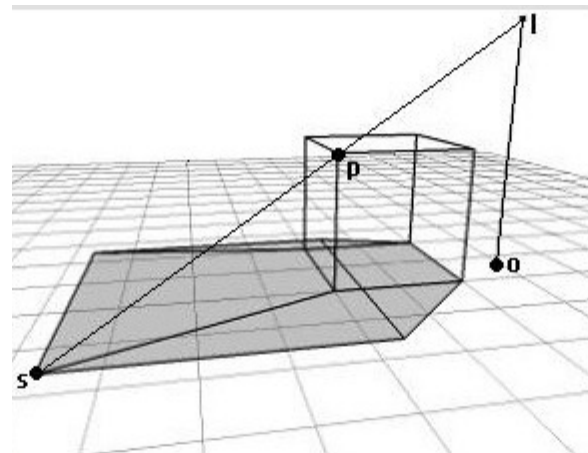
- Ombre su piani (proiezione)
- Shadow Mapping (Shadow Buffer/Map)
- Shadow Volumes (non li vedremo)

Ombre su piani (Blinn 1988)

Obiettivo: generare una geometria piatta per proiezione sul piano, quindi disegnarla con colore ombra.

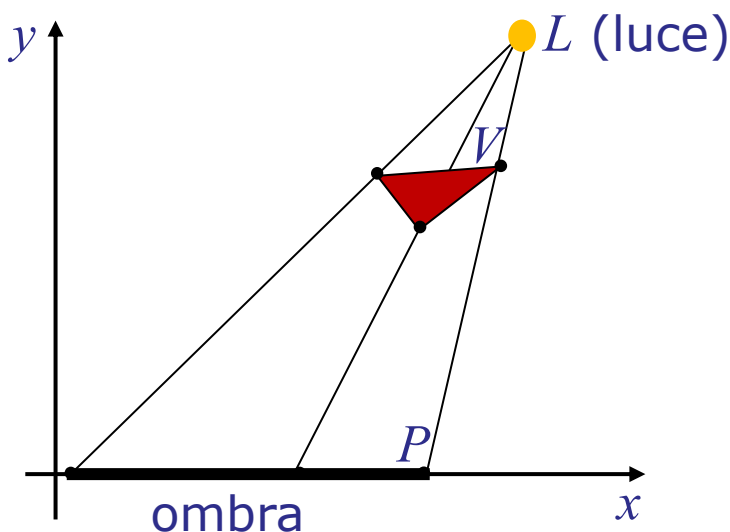
Algoritmo: Proiezione di un oggetto su un piano (pavimento/parete)

- Si costruisce la matrice di proiezione dalla luce al piano
- Si applica tale matrice alla geometria 3D di cui si vuole ottenere l'ombra
- Si disegna l'oggetto piatto con colore ombra



Ombre su piani

Caso speciale: l'ombra viene generata su un piano cartesiano; basta proiettare i vertici su quel piano e considerare l'oggetto piatto formato dalle facce proiettate.



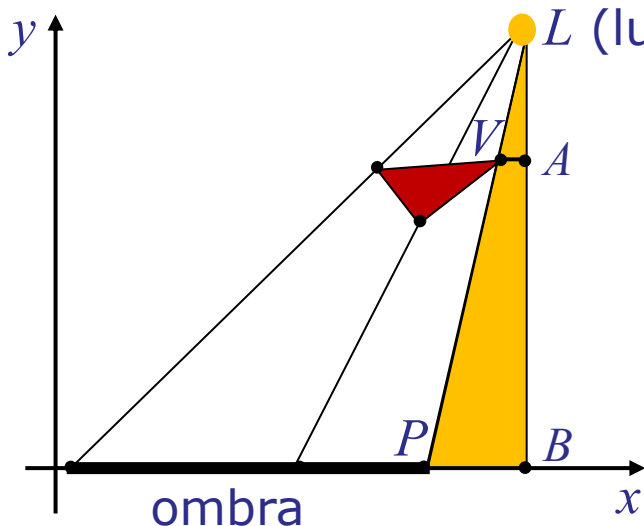
Dati:

- posizione della luce: L
- posizione del piano: $y=0$
- posizione del vertice: V

Calcolare la proiezione P di V sul piano $y=0$.

Ombre su piani

Usiamo la similitudine fra i triangoli PBL e VAL per determinare P :



$$(Lx - Px) : (Lx - Vx) = Ly : (Ly - Vy)$$

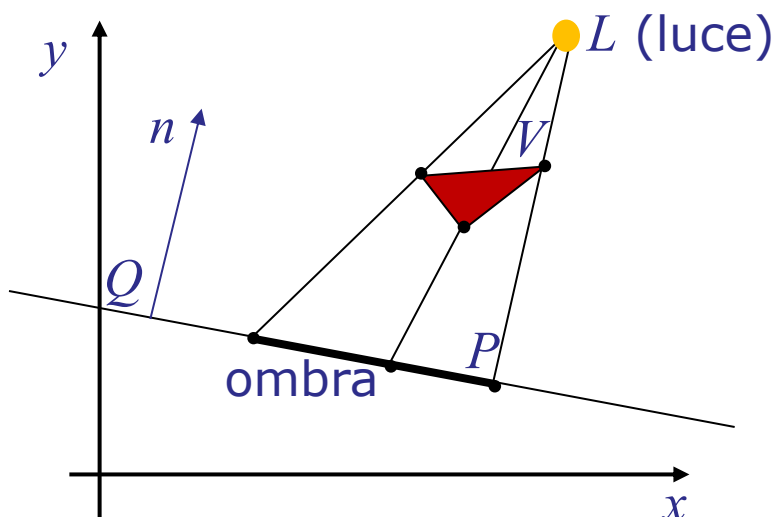
$$\frac{(Px - Lx)}{(Vx - Lx)} = \frac{Ly}{(Ly - Vy)}$$

$$Px - Lx = \frac{Ly (Vx - Lx)}{(Ly - Vy)}$$

$$Px = \frac{Ly Vx - Lx Vy}{(Ly - Vy)}$$

Ombre su piani

Caso generale: l'ombra viene generata su un piano arbitrario; bisogna proiettare i vertici su quel piano e considerare l'oggetto piatto formato dalle facce proiettate.



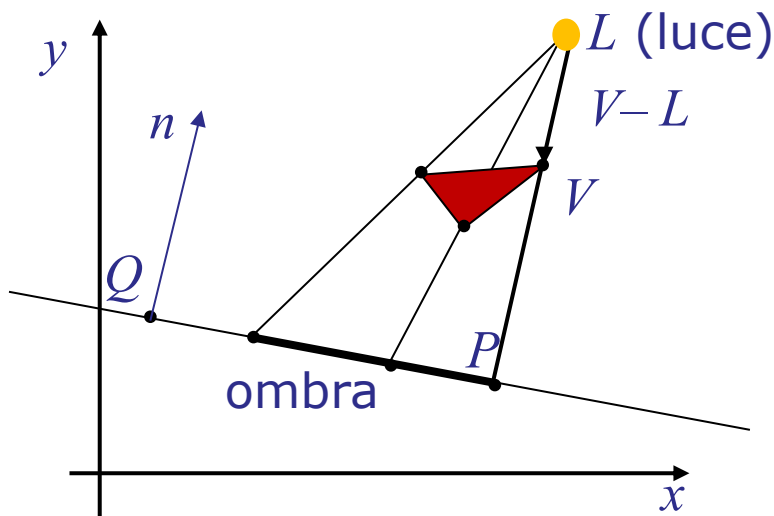
Dati:

- posizione della luce: L
- un punto sul piano: $Q = [q_x, q_y, q_z]$
- normale al piano: $n = [a, b, c]$
- posizione del vertice: V

Equazione del piano: $ax + by + cz + d = 0$
con $d = -a q_x - b q_y - c q_z$

Calcolare la proiezione P di V sul piano $ax + by + cz + d = 0$

Ombre su piani



Dati:

- posizione della luce: L
- un punto sul piano: $Q = [q_x, q_y, q_z]$
- normale al piano: $n = [a, b, c]$
- posizione del vertice: V

Equazione del piano: $ax + by + cz + d = 0$
con $d = -a q_x - b q_y - c q_z$

Sarà: $P = L + \text{scal} (V - L)$

Sappiamo che: $n \cdot P + d = 0$, perché P deve stare sul piano; allora per trovare il valore dello scalare scal possiamo risolvere:

$$n \cdot (L + \text{scal} (V - L)) + d = 0$$

$$n \cdot L + \text{scal} n \cdot (V - L) + d = 0$$

$$\text{scal} = - \frac{d + n \cdot L}{n \cdot (V - L)}$$



Ombre su piani in WebGL

Luce: $L = [L_1, L_2, L_3, L_4]$ in coord. omogenee

Normale al piano: $n = [n_1, n_2, n_3]$

Punto sul Piano: $Q = [q_1, q_2, q_3]$

$$d = -n_1 q_1 - n_2 q_2 - n_3 q_3$$

$$\text{Sia } dot = n_1 L_1 + n_2 L_2 + n_3 L_3 + d L_4$$

$$M = dot \, I_{4 \times 4} - \begin{bmatrix} n_1 L_1 & n_1 L_2 & n_1 L_3 & n_1 L_4 \\ n_2 L_1 & n_2 L_2 & n_2 L_3 & n_2 L_4 \\ n_3 L_1 & n_3 L_2 & n_3 L_3 & n_3 L_4 \\ d L_1 & d L_2 & d L_3 & d L_4 \end{bmatrix}$$

$$P = M V$$



Ombre su piani: teoria

Sia $r = \alpha V + \beta L$ con $\alpha + \beta = 1$

allora $P = \alpha V + \beta L$ (1) per opportuni α e β

Normale al piano: $n = [n_1, n_2, n_3]$

Punto sul Piano: $Q = [q_1, q_2, q_3]$

$$d = -n_1 q_1 - n_2 q_2 - n_3 q_3$$

allora $n \cdot P + d = 0$ od anche $[n_1, n_2, n_3, d][p_1, p_2, p_3, p_4]^T = 0$ (2)

Dobbiamo combinare la (1) e la (2) e trovare α e β che soddisfino la (2); c'è un modo elegante per fare questo:

$$\alpha = n \cdot L$$

$$\beta = -n \cdot V$$

$$P = (n \cdot L) V - (n \cdot V) L$$

Ora non resta da fare altro che scrivere quest'ultima considerando le sue componenti, per avere una espressione matriciale in coordinate omogenee.



Ombre su piani: teoria

$$P_i = \sum_j n_j L_j V_i - \sum_j n_j V_j L_i \quad i=1,2,3,4$$

$$M_{ij} = -n_j L_i + \begin{cases} \alpha & \text{se } i=j \\ 0 & \text{altrimenti} \end{cases} \quad i, j=1,2,3,4$$

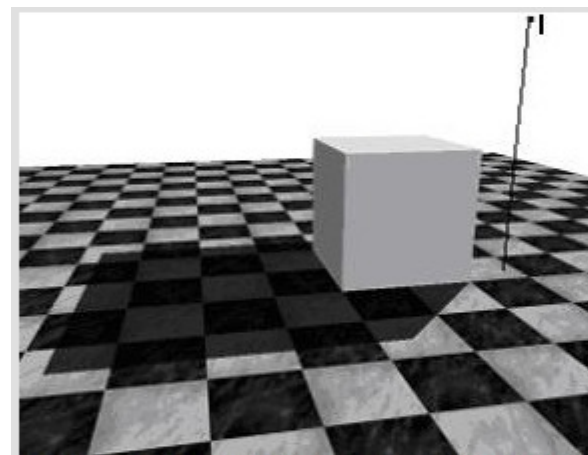
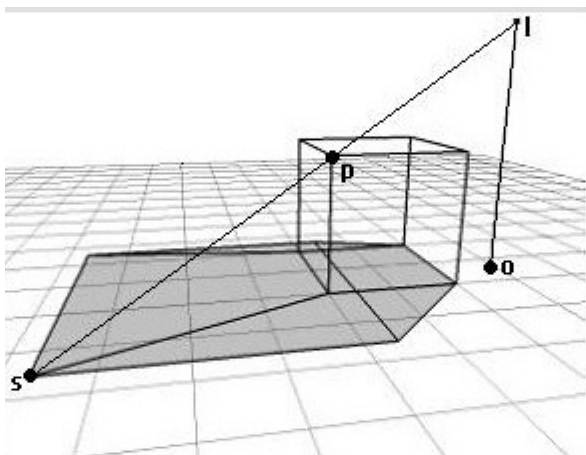
$$M = \begin{bmatrix} \alpha - n_1 L_1 & -n_1 L_2 & -n_1 L_3 & -n_1 L_4 \\ -n_2 L_1 & \alpha - n_2 L_2 & -n_2 L_3 & -n_2 L_4 \\ -n_3 L_1 & -n_3 L_2 & \alpha - n_3 L_3 & -n_3 L_4 \\ -d L_1 & -d L_2 & -d L_3 & \alpha - d L_4 \end{bmatrix}$$

$$P = M V$$

Ombre su piani

Vantaggi e Svantaggi:

- Veloce e semplice
- Funziona solo per superfici piane
- Non funziona per auto-ombre





Esempio di Ombre su piani

Esaminare il seguente codice di esempio nella cartella
HTML5_webgl_3

cube_shadows_GUI.html e .js

Esercizio: modificare il codice affinché vengano rappresentati la luce nella sua posizione e un piano su cui si stagli l'ombra.



Shadow Mapping

Il più semplice algoritmo per le ombre, facilmente integrabile in uno Z-Buffer, è lo **Shadow Mapping** sviluppato da **Williams nel 1978**.

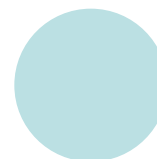
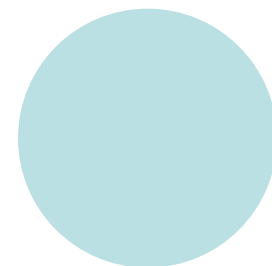
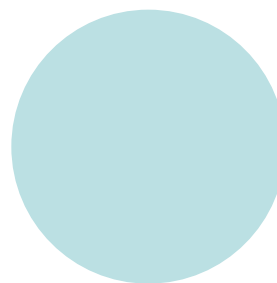
Questo metodo richiede un buffer per le ombre (**Shadow Buffer**), diverso dallo Z-Buffer, per ogni sorgente luminosa.

Nella sua forma base, è efficiente solo per scene illuminate da una sola sorgente puntiforme. In alternativa, si può usare un solo buffer per le ombre anche per più luci, ma l'algoritmo deve essere eseguito per ogni luce, e questo lo rende più pesante.

L'algoritmo consiste in un processo a due passi.

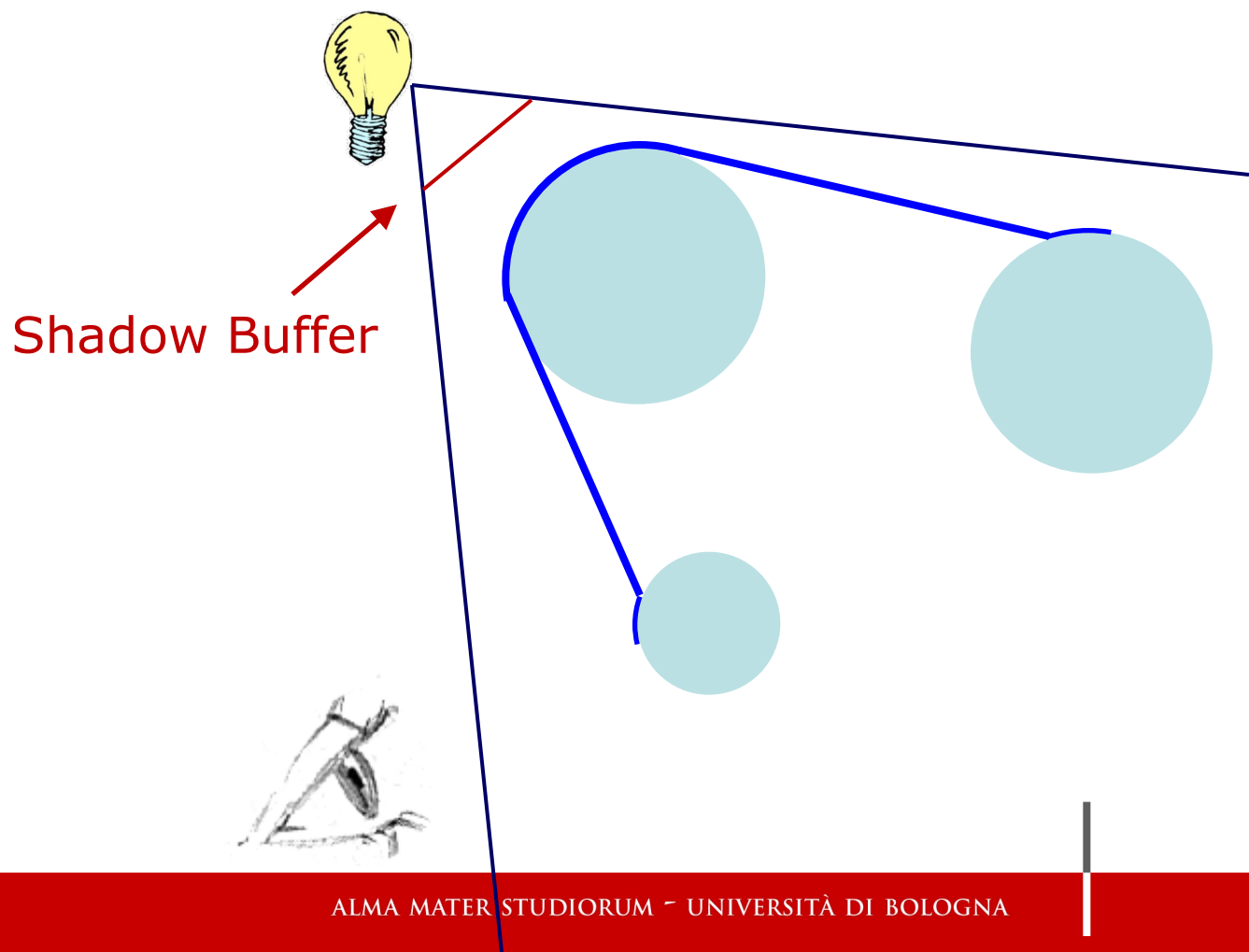


Shadow Mapping



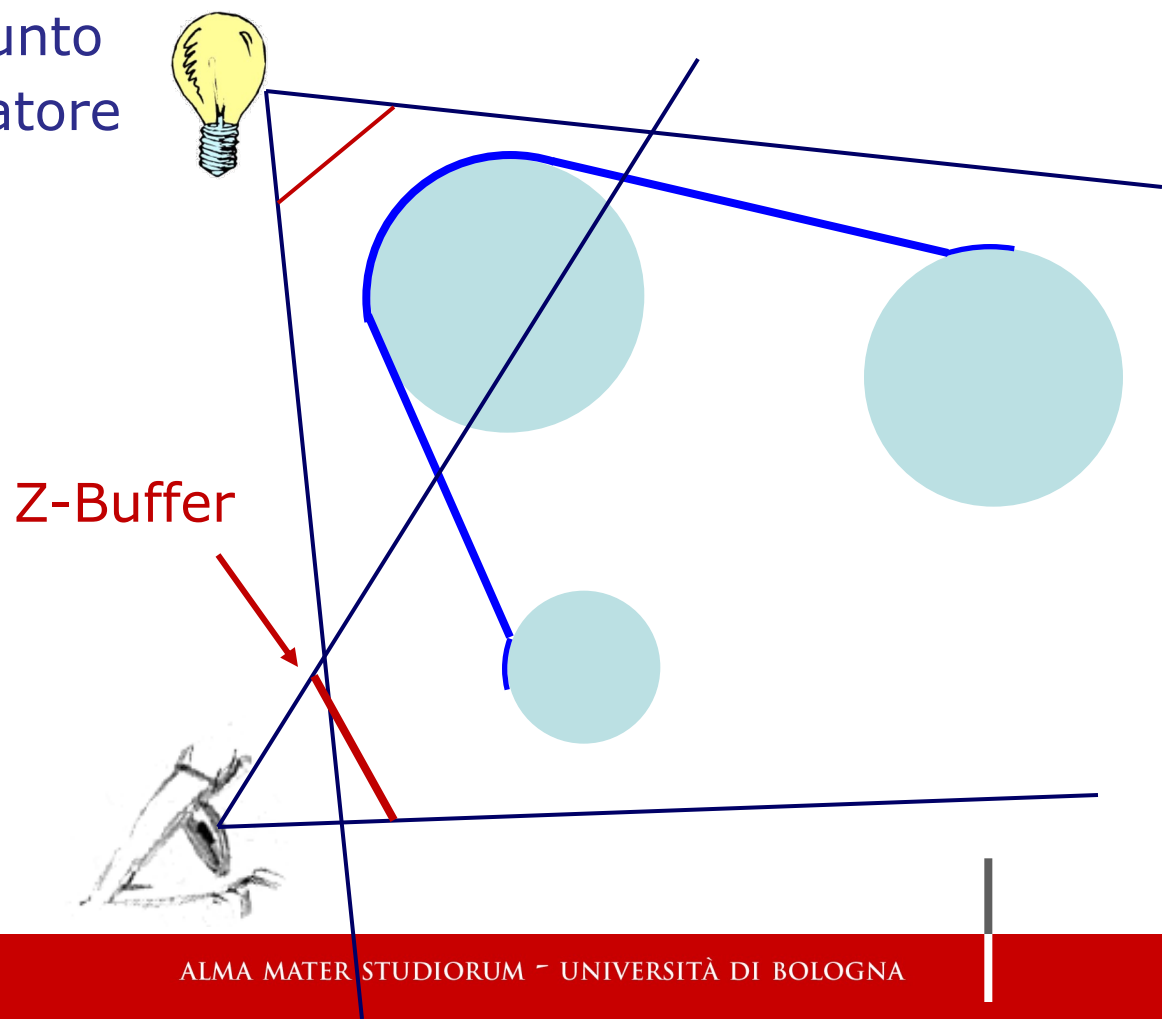
Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa



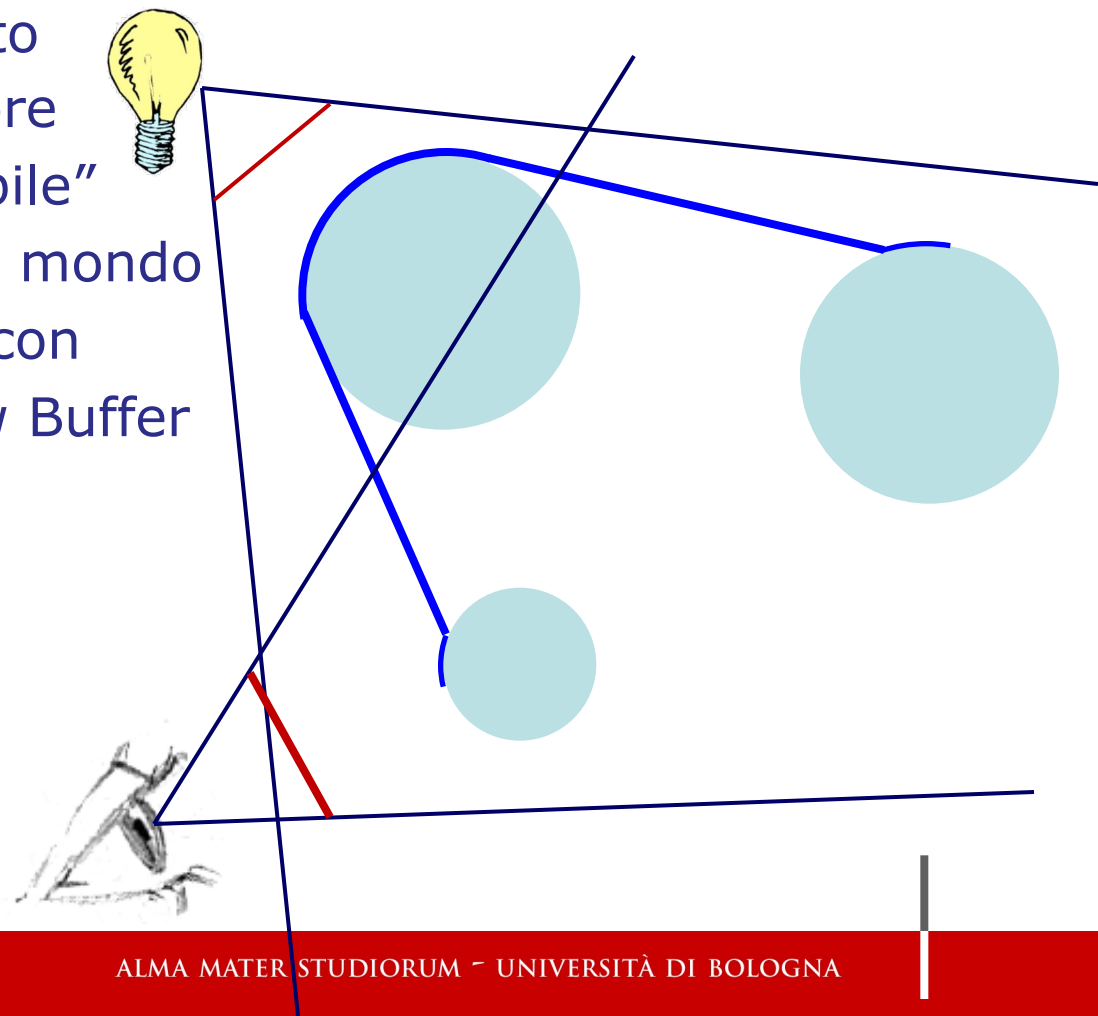
Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa
- Rende la scena dal punto di vista dell'osservatore



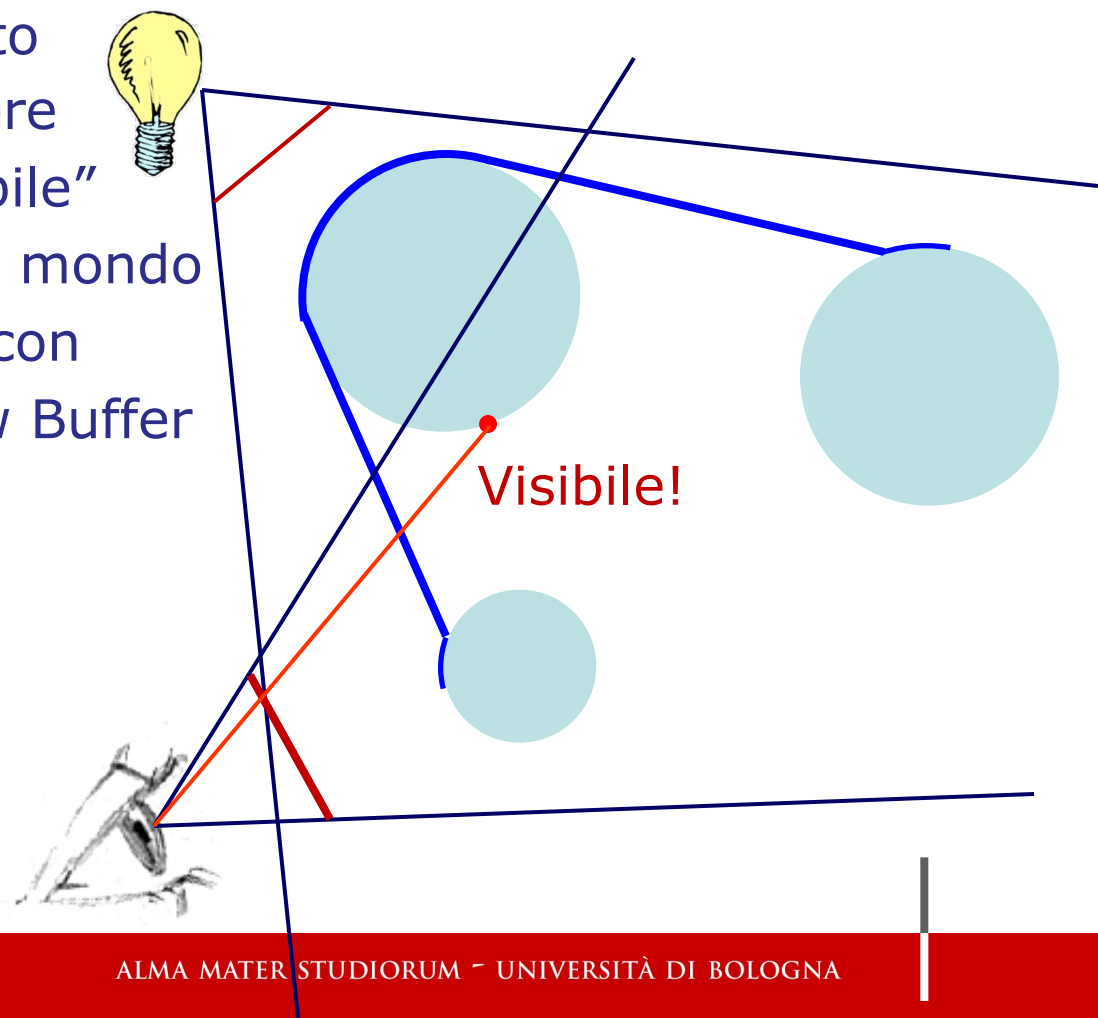
Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa
- Rende la scena dal punto di vista dell'osservatore
- Per ogni fragment "visibile"
 - trasforma nello spazio mondo
 - confronta la distanza con il valore nello Shadow Buffer



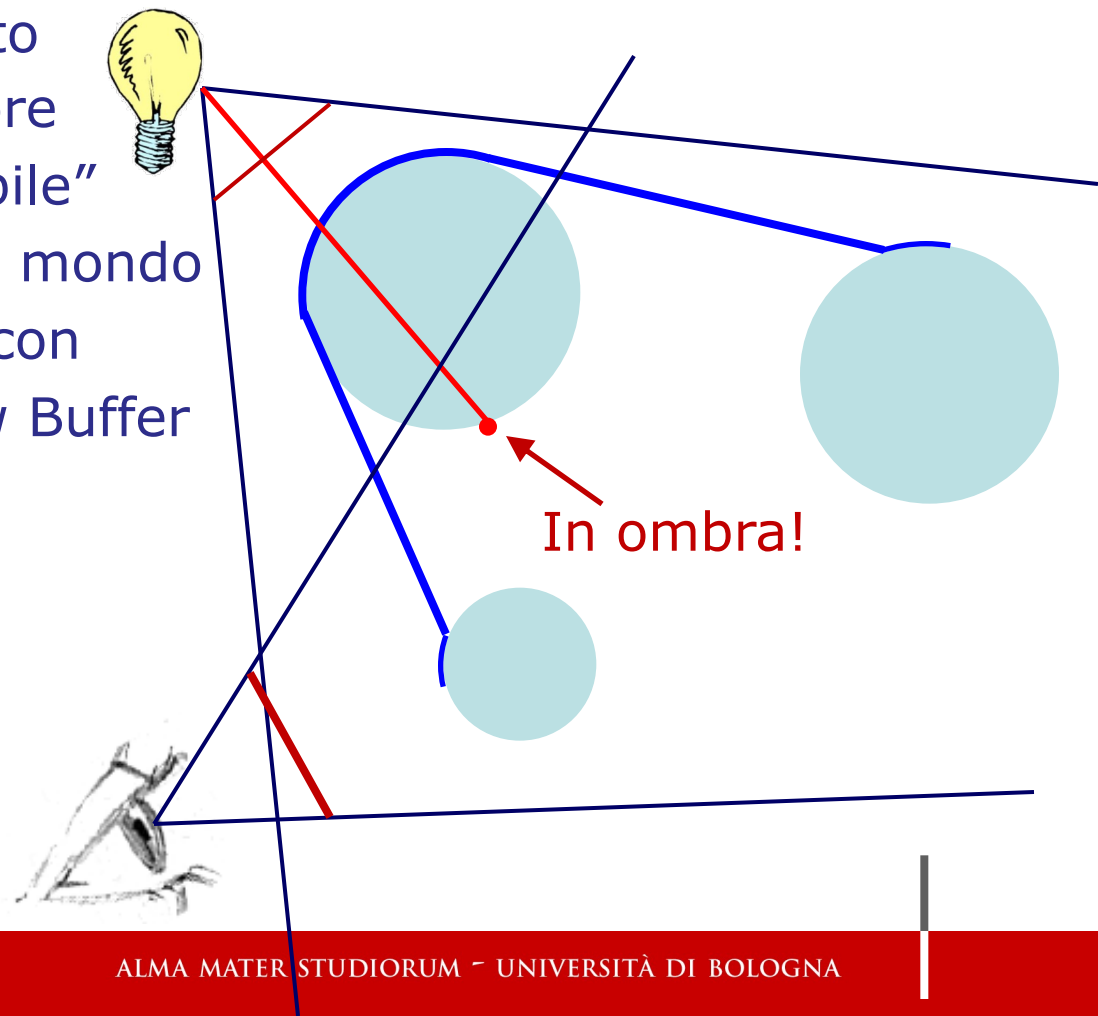
Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa
- Rende la scena dal punto di vista dell'osservatore
- Per ogni fragment "visibile"
 - trasforma nello spazio mondo
 - confronta la distanza con il valore nello Shadow Buffer



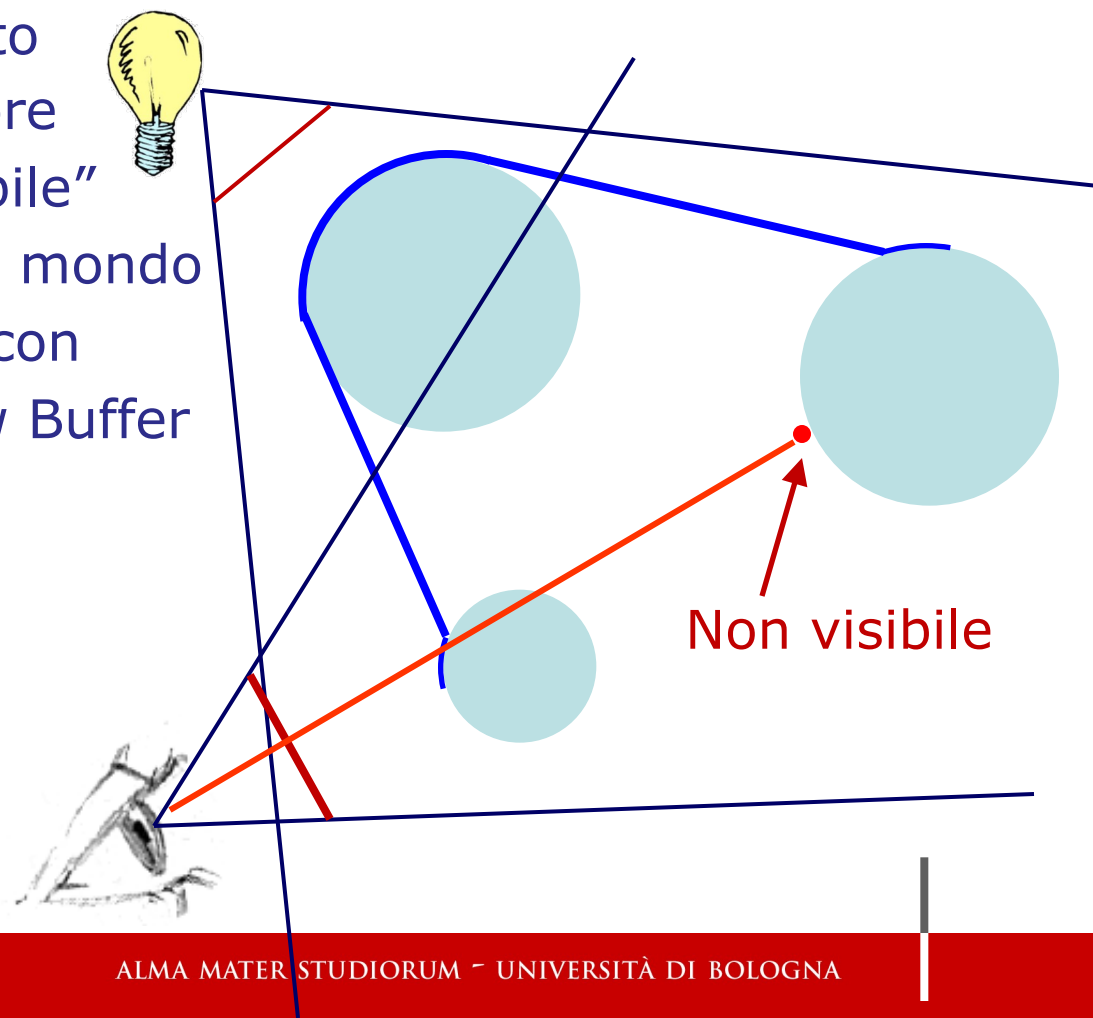
Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa
- Rende la scena dal punto di vista dell'osservatore
- Per ogni fragment "visibile"
 - trasforma nello spazio mondo
 - confronta la distanza con il valore nello Shadow Buffer



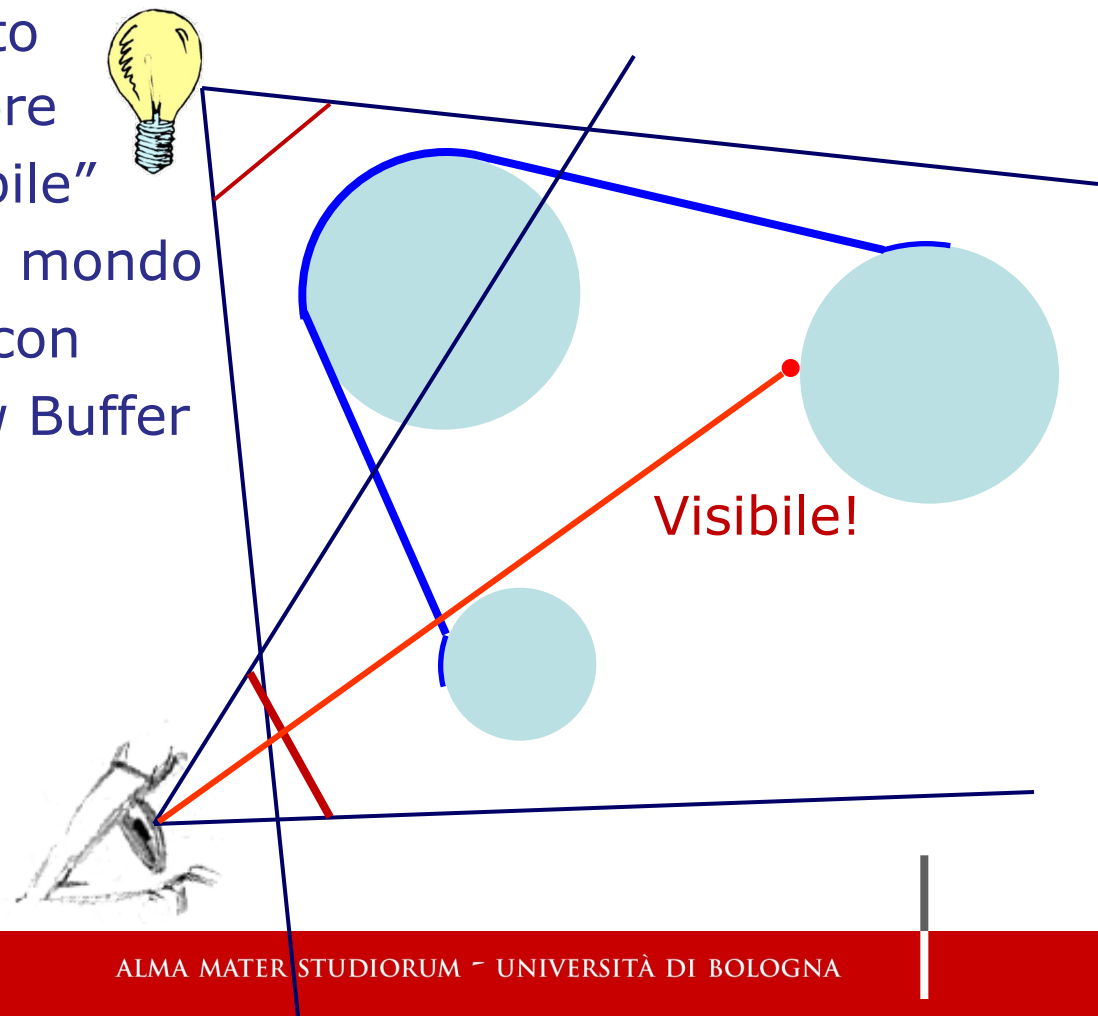
Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa
- Rende la scena dal punto di vista dell'osservatore
- Per ogni fragment "visibile"
 - trasforma nello spazio mondo
 - confronta la distanza con il valore nello Shadow Buffer



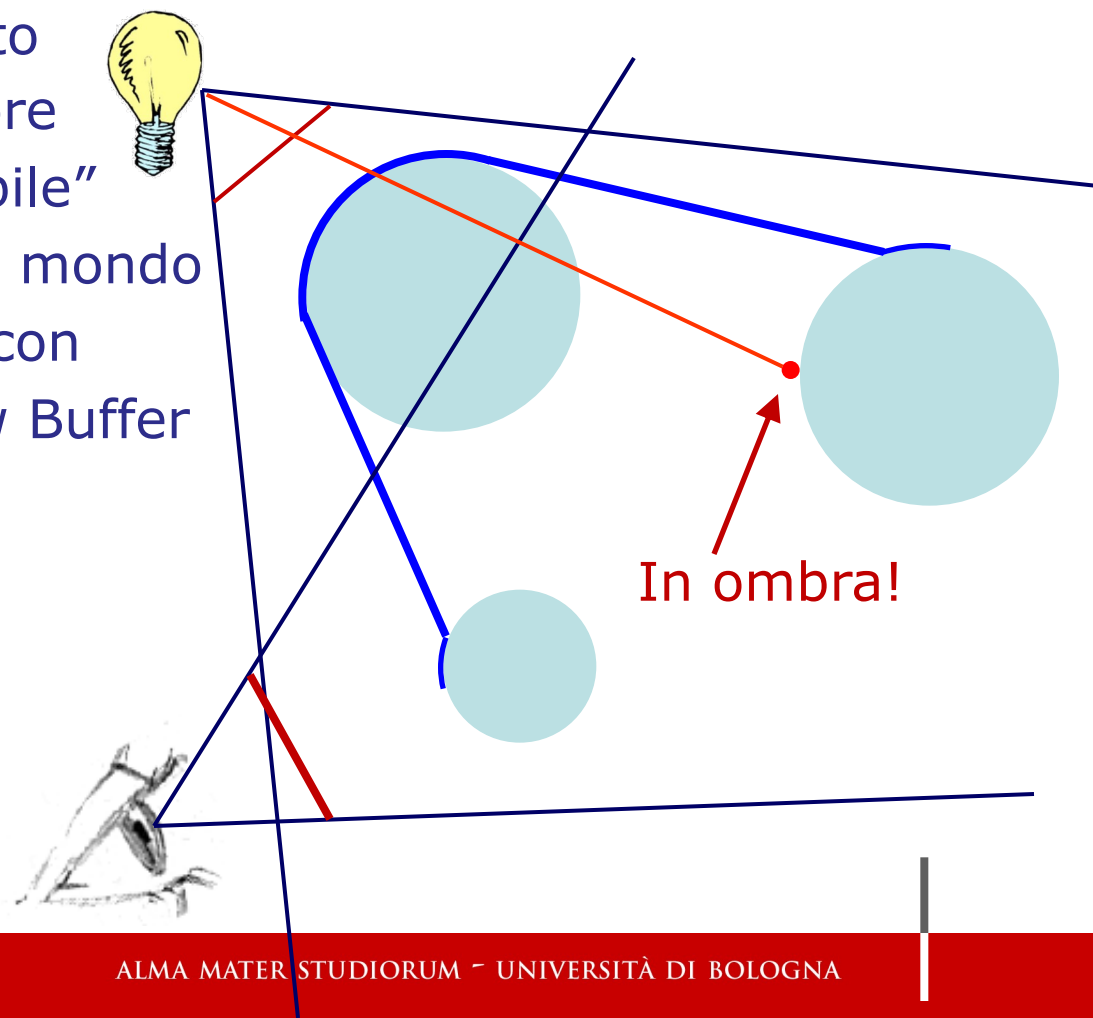
Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa
- Rende la scena dal punto di vista dell'osservatore
- Per ogni fragment "visibile"
 - trasforma nello spazio mondo
 - confronta la distanza con il valore nello Shadow Buffer



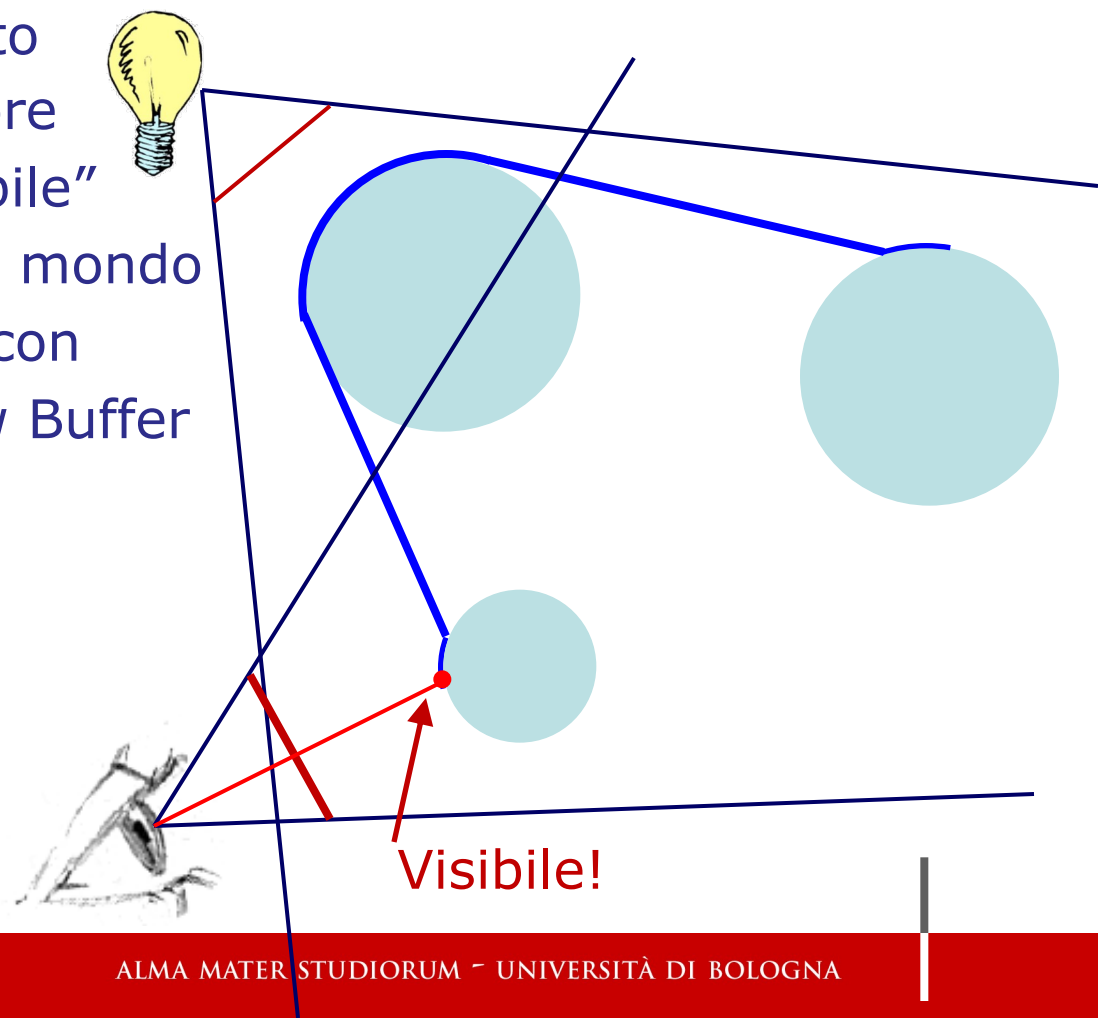
Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa
- Rende la scena dal punto di vista dell'osservatore
- Per ogni fragment "visibile"
 - trasforma nello spazio mondo
 - confronta la distanza con il valore nello Shadow Buffer



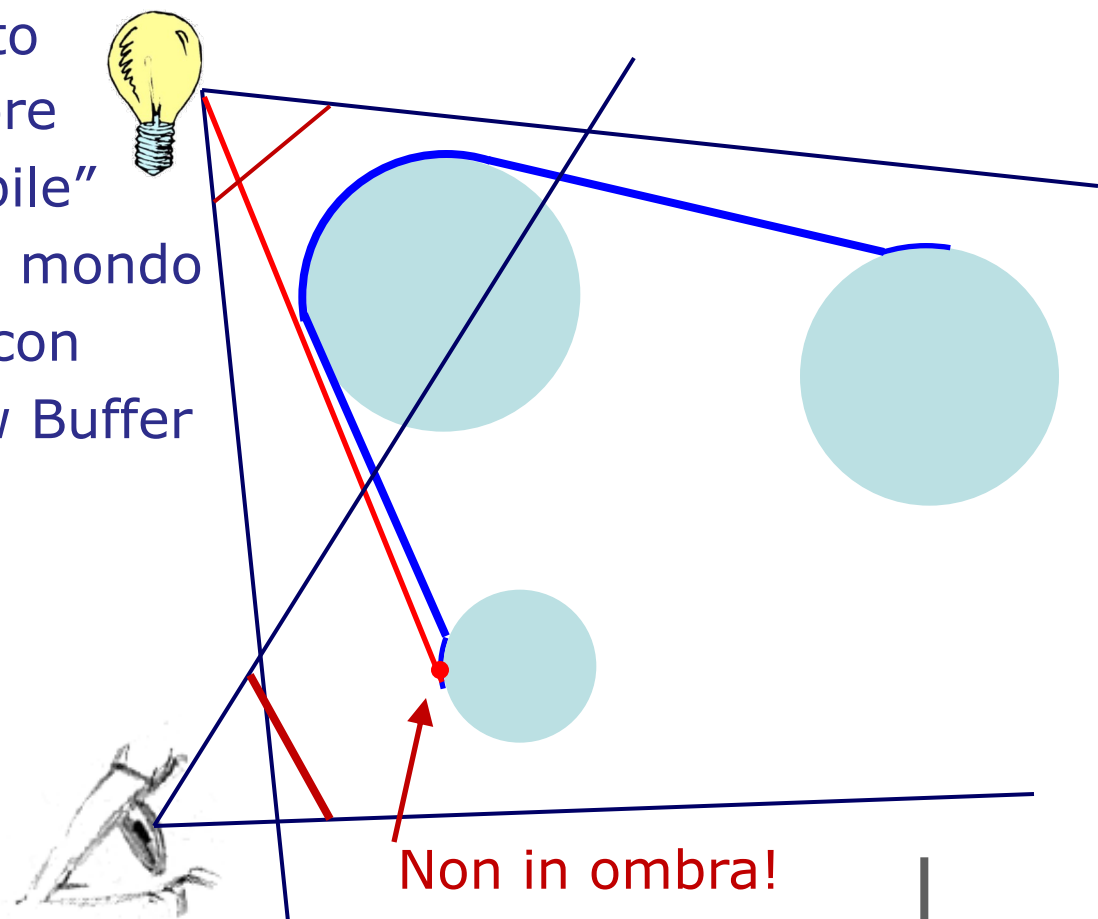
Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa
- Rende la scena dal punto di vista dell'osservatore
- Per ogni fragment "visibile"
 - trasforma nello spazio mondo
 - confronta la distanza con il valore nello Shadow Buffer



Shadow Mapping

- Rende la scena dal punto di vista della sorgente luminosa
- Rende la scena dal punto di vista dell'osservatore
- Per ogni fragment "visibile"
 - trasforma nello spazio mondo
 - confronta la distanza con il valore nello Shadow Buffer





Shadow Mapping

1. Si considera come punto di vista la sorgente luminosa e si applica un classico depth-buffer, ma non si renderizza nulla e le profondità vengono memorizzate nello Shadow Buffer;

2. Si applica lo Z-Buffer per rendere la scena dal punto di vista dell'osservatore, modificato come segue:

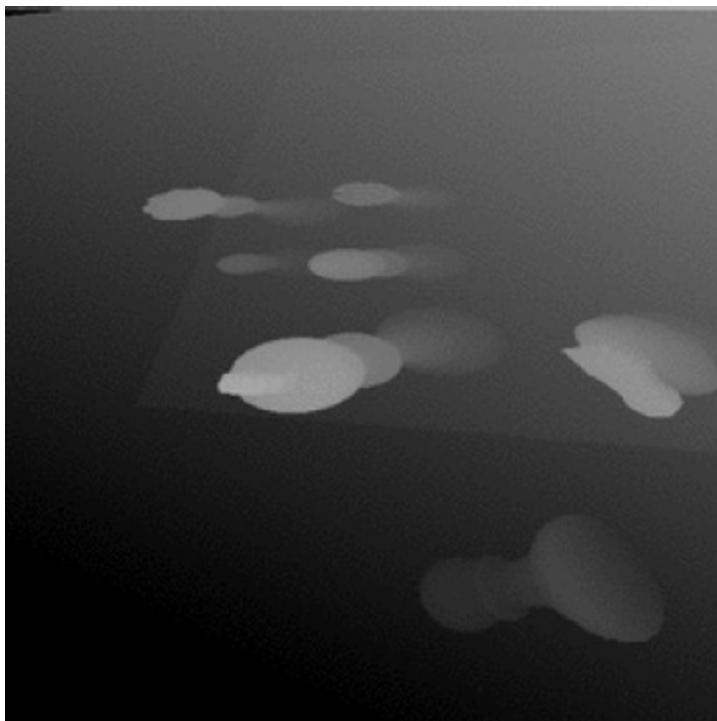
- se un fragment/pixel è visibile, si applica una trasformazione per mappare le coordinate $[x, y, z]$ del punto nello spazio schermo (dal punto di vista) in $[x', y', z']$ del punto nello spazio schermo (dalla sorgente luminosa).
- sia z_s il valore nello Shadow Buffer nella posizione $[x', y']$;
- se $z' > z_s$ il punto sarà in ombra e sarà reso con intensità di ombra, altrimenti il punto verrà reso normalmente.



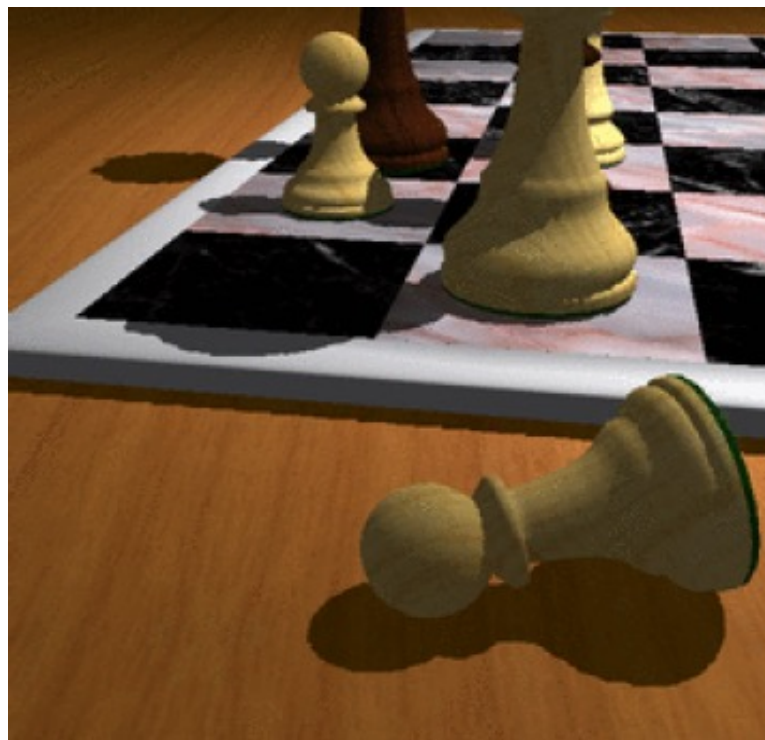
Shadow Mapping: Algoritmo

- Rendere la scena come se fosse vista dalla sorgente luminosa
- Salvare il depth dei pixel (Shadow Buffer)
- Rendere la scena dalla posizione dell'osservatore
 - Trasformare le coordinate del pixel nel sistema di coordinate della luce
 - Confrontare z' con il valore z_s memorizzato nella corrispondente posizione nello Shadow Buffer
 - Il pixel è in ombra se $z' > z_s$

Shadow Mapping: Esempio

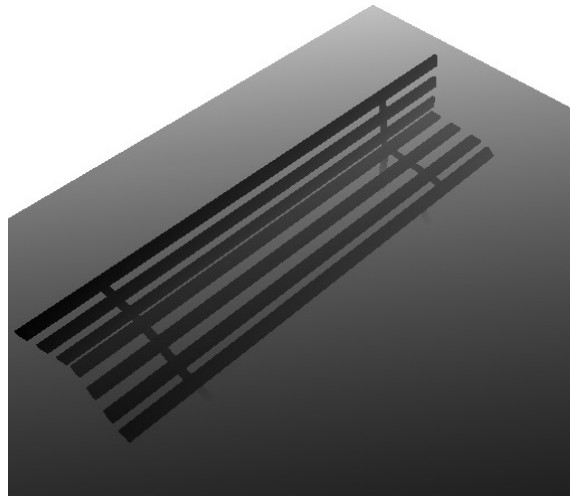


Shadow map

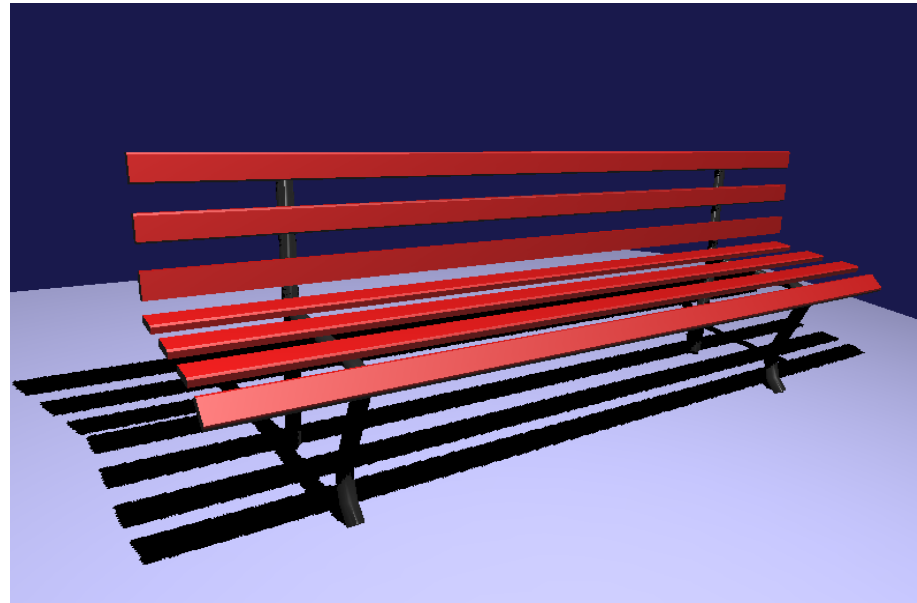


Scena con una luce

Shadow Mapping: Esempio

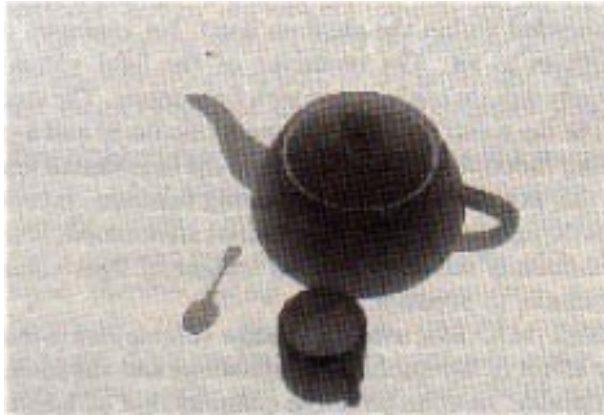


Shadow map



Scena con una luce

Shadow Mapping: Esempio



Shadow map



Scena con due luci



Shadow Mapping

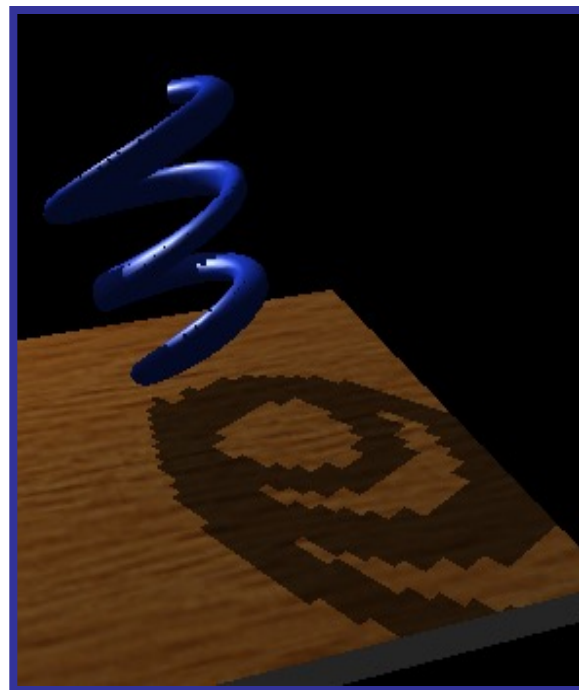
- Vantaggi
 - Semplice da implementare
 - Non dipende dalla complessità della scena (tranne che per rendere lo Shadow Buffer)
- Svantaggi
 - Immagini a risoluzione fissa portano ad artefatti

Aliasing nello Z-Buffer con ombre mediante Shadow Mapping

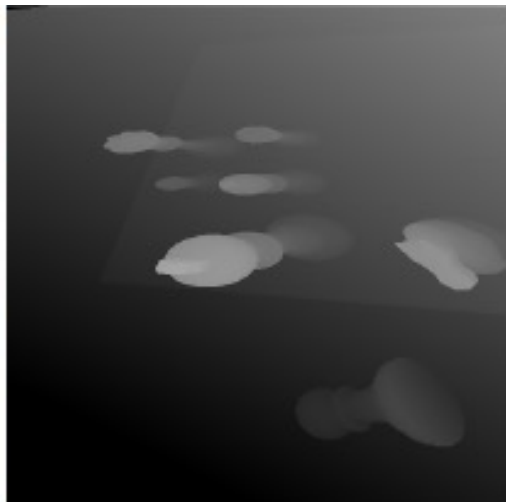
Con aliasing ci si riferisce ad un effetto che comporta che differenti segnali, quando vengono campionati, siano indistinguibili (da alias uno al posto dell'altro).

Ci sono due tipi di aliasing:

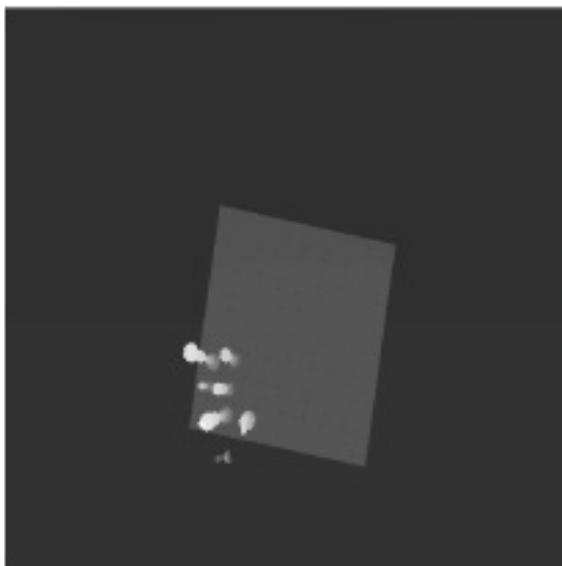
- lungo i lati delle ombre (che come detto sono ben nette a causa di luci puntiformi) si notano le tipiche scalettature;
- quando si proietta un pixel nel Buffer delle ombre, cioè su un discreto, si deve effettuare una decisione netta (analogo a quanto visto per le texture).



Shadow Mapping: Esempio



Shadow map





Shadow Mapping con WebGL

Renderizza la scena dal punto di vista della luce e salva le distanze in un buffer. Poi, nel rendering finale, confronta la distanza del pixel con quella nel buffer per capire se è in ombra.

WebGL ha un **Depth Buffer**, ma la GPU lo usa internamente per capire quali oggetti sono davanti ad altri (Z-buffering); l'utente non può accedere a quei dati.

L'estensione **WEBGL_depth_texture** permette di leggere i dati del **Depth Buffer** come se fosse una normale texture.

Abilitando l'Extension, si può creare una texture e anziché scriverci i colori (RGBA) ci si può, per esempio, scrivere quanto ogni pixel è lontano dalla camera.



Shadow Mapping con WebGL

Questo permette effetti grafici avanzati come:

1. **Shadow Mapping** (è l'uso principale)
2. Effetti di Post Processing:
 1. **Depth of Field**: sfocare gli oggetti in base alla loro distanza (come una macchina fotografica reale).
 2. **Nebbia (Fog)**: creare nebbia volumetrica basata sulla profondità reale dei pixel.
 3. **Screen Space Reflections (SSR)**: riflessi realistici calcolati in base alla posizione 3D dei pixel sullo schermo.
3. **Ottimizzazione**: si può usare la texture di profondità per calcoli di collisione o per evitare di renderizzare pixel già coperti da altro.



WebGL_depth_texture

Come si attiva questa WebGL Extension?

Trattandosi di un'estensione (standard in WebGL 2) opzionale in WebGL 1, la si deve richiedere esplicitamente:

```
const depthExt = gl.getExtension('WEBGL_depth_texture');  
if (!depthExt) {  
    console.error("Il browser non supporta le texture di profondità!");  
}
```

Quando si crea la texture con questa estensione, si devono usare dei formati specifici nella funzione **gl.texImage2D**:

- **internalFormat**: gl.DEPTH_COMPONENT
- **format**: gl.DEPTH_COMPONENT
- **type**: gl.UNSIGNED_SHORT o gl.UNSIGNED_INT_24_8_WEBGL



Shadow Map

Vediamo un esempio per configurare un **Framebuffer (FBO)** che scrive i dati di profondità in una texture, invece che in un normale buffer di rendering. Questo è il primo passo per creare le **Shadow Map**.

1. Attivare l'estensione (solo per WebGL 1)

```
const ext = gl.getExtension('WEBGL_depth_texture');  
if (!ext) {alert("Il browser non supporta le texture di profondità!");}
```

2. Creare la texture di profondità

```
const depthTexture = gl.createTexture();  
const size = 1024; // Risoluzione della mappa delle ombre
```

```
gl.bindTexture(gl.TEXTURE_2D, depthTexture);
```

...



Shadow Map

```
gl.texImage2D(  
    gl.TEXTURE_2D,          // target  
    0,                      // level  
    gl.DEPTH_COMPONENT,    // internalFormat (IMPORTANTE)  
    size, size,             // width, height  
    0,                      // border  
    gl.DEPTH_COMPONENT,    // format  
    gl.UNSIGNED_INT,        // type (puoi usare anche gl.UNSIGNED_SHORT)  
    null                    // dati (null perché la riempirà la GPU)  
);
```

```
// Filtri necessari per le texture di profondità  
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.NEAREST);  
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.NEAREST);  
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE);  
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.CLAMP_TO_EDGE);
```



Shadow Map

3. Creare il Frame Buffer e collegarla

```
const fb = gl.createFramebuffer();  
gl.bindFramebuffer(gl.FRAMEBUFFER, fb);  
  
// Collega la texture al canale della profondità  
gl.framebufferTexture2D(  
    gl.FRAMEBUFFER,  
    gl.DEPTH_ATTACHMENT, // Specifichiamo che è per la profondità  
    gl.TEXTURE_2D,  
    depthTexture,  
    0  
);  
  
// Nota: Molte GPU richiedono comunque una texture di colore "vuota"  
// o di disabilitare esplicitamente il colore per far funzionare l'FBO.
```



Shadow Map

Quindi, quando si vuole generare la Shadow Map:

1. Fare il bind del framebuffer:

```
gl.bindFramebuffer(gl.FRAMEBUFFER, fb);
```

2. Impostare il viewport alla dimensione della texture:

```
gl.viewport(0, 0, size, size);
```

3. Pulire solo il depth buffer:

```
gl.clear(gl.DEPTH_BUFFER_BIT);
```

4. Disegnare la scena dal punto di vista della luce.

Dopo queste operazioni, **depthTexture** conterrà la distanza di ogni oggetto dalla luce.

Nel passaggio successivo (il rendering finale), si passa questa texture allo shader per confrontare le profondità e decidere se un pixel è "coperto" da un altro oggetto.



Shadow Acne

L'**acne** da ombra si manifesta come delle fastidiose **strisce nere** o **artefatti** sulla superficie che dovrebbe essere illuminata.

Accade perché la **Shadow Map** ha una risoluzione finita (è fatta di pixel, o texel).

Quando la GPU confronta la profondità di un punto sulla superficie con il valore salvato nella shadow map:

- a causa di piccoli errori di precisione numerica (floating point), un punto può risultare leggermente dietro se stesso;
- la GPU pensa che la superficie si stia facendo ombra da sola, disegnando pixel neri alternati.



La soluzione: il Bias

Il **bias** consiste nel sottrarre un piccolo valore alla profondità calcolata (o aggiungerlo alla profondità della mappa) prima di fare il confronto.

In pratica, si chiede alla GPU di *considerare il punto in ombra solo se è significativamente più lontano rispetto a quello della shadow map*.

Effetto collaterale: Peter Panning

Non si può esagerare con il bias. Se il valore è troppo alto, l'ombra si "stacca" dall'oggetto, facendolo sembrare sospeso in aria; quindi:

- **Bias troppo piccolo:** Shadow Acne (strisce nere).
- **Bias troppo grande:** Peter Panning (ombra volante).



Esempio su Shadow Mapping

Esaminare il seguente codice di esempio nella cartella
HTML5_webgl_3

sphere_shadow_mapping_GUI.html e .js

Che utilizza l'estensione WEBGL_depth_texture

Vedere le estensioni di WebGL in:

<https://registry.khronos.org/webgl/extensions/>

Luxo Jr. in Real-time using Shadow Mapping

Luxo Jr. Demo Details

Steve Jobs at 2001 MacWorld Japan shows this on a Mac with OpenGL using hardware shadow mapping

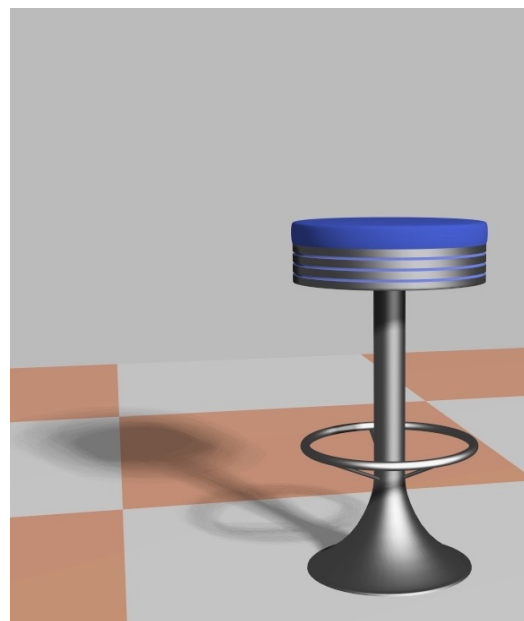
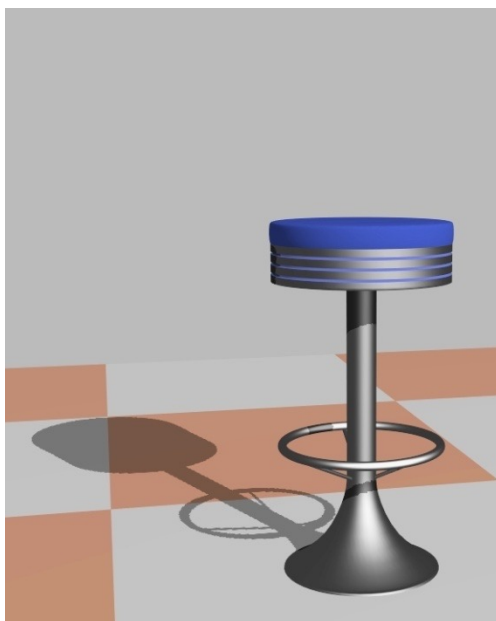
- Luxo Jr. has two animated lights and one overhead light
 - Three shadow maps *dynamically* generated per frame
- Complex geometry (cords and lamp arms) all correctly shadowed
- User controls the view, shadowing just works
- Real-time Luxo Jr. is technical triumph for OpenGL
- Only available in OpenGL.



see youtube: Apple MacWorld Japan 2001 Luxo Jr. Demo
<https://www.youtube.com/watch?v=nOmgUR4L8I0>

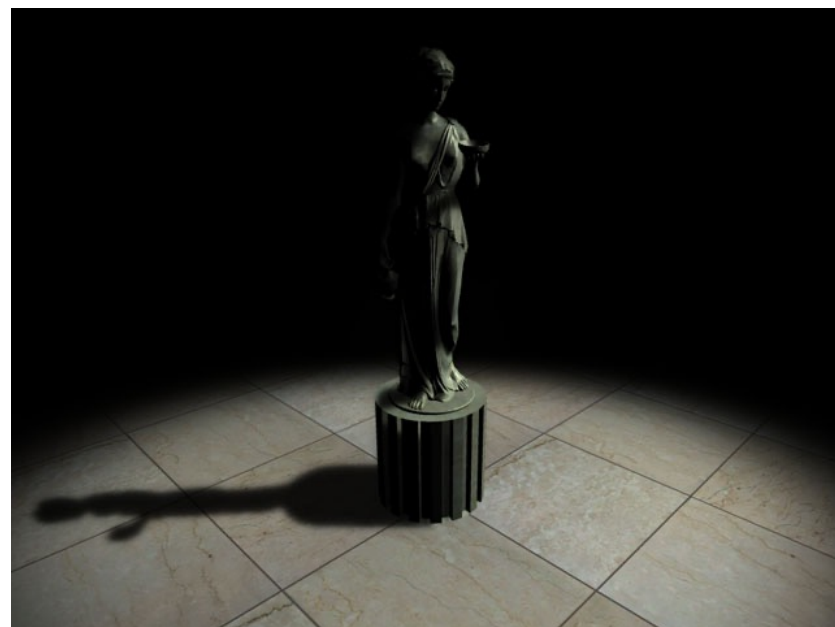
Soft Shadow

- La luce puntiforme causa, come già detto, hard shadow
- Le sorgenti puntiformi, in realtà, non sono infinitamente piccole
- Le sorgenti di luce hanno una certa area o volume e producono soft shadow



Soft Shadow

- Simulare un'area di luce usando più sorgenti puntiformi (può essere costoso in tempo)
- Nello spazio immagine effettuare un blur delle ombre
- Utilizzare una tecnica di Antialiasing per le ombre





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Giulio Casciola
Dip. di Matematica
giulio.casciola@unibo.it